Solving Stochastic Differential Equations

R

April 23, 2026

Contents

1 Target SDE **1**

2 Implementing a Normal Distribution Function **2**

3 Numpy Accelerating the Euler Maruyama Approximation **2**
3.1 Performance Timing and Accuracy Estimation 4

4 Defining the Runge-Kutta approximation **5**

5 Pricing Exotic Options **7**

6 Advanced Monte Carlo to solve path-dependent options **10**

Contents

1 Target SDE

We aim to experimentally obtain expected values for a `Seasonal Volatility Model` SDE:

$$dS_t = S_t \mu dt + S_t \sigma [1 + 0.9 \sin(2\pi t)] dW_t$$

This can be described as an Itô process:

$$dS_t = a(t, S_t) d_t + b(t, S_t) dW_t,\ X_0 = x$$

Where:

$$a(t, S_t) = S_t \mu$$

$$b(t, S_t) = S_t \sigma [1 + 0.9 \sin(2\pi t)]$$

We can prototype a simple iterative differencing algorithm to approximate an n-th step term based on the Euler-Maruyama scheme:

$$\tilde{X}_{n+1} = \tilde{X}_n + a(t_n, \tilde{X}_n) \delta t + b(t_n, \tilde{X}_n) \Delta W_n$$

Where we will break down the time interval into small blocks, indexed to become our Δt, and generate normal variables for each interval, becoming our ΔW_n.

```
def price_euler (S_0, mu, sigma, t0, T, N):
    Declare dt as T / N
    Declare W_0 as 0

    for i from 0 up to (not including) N:
        sample the z-distribution N+1 times
        scale the z vector by sqrt(dt)
        take differences of the z vector

    for i from 0 up to (not including) N:
        calculate the process using dt and dZ

    return S_0,...,S_n
```

Listing 1: Basic Euler-Maruyama implementation

2 Implementing a Normal Distribution Function

We need a way to sample from the normal distribution. An immediate solution is to leverage the C++ STL, which includes a function that provides uniform distribution estimates over a continuous interval. We can apply the inverse quantile transform to convert what becomes a probability distribution over $[0, 1]$ into standard normal distribution draws, which can easily be converted into any normal distribution. Abramowitz and Stegun suggest the approximation:

$$x_p = t - \frac{c_0 + c_1 t + c_2 t^2}{1 + d_1 t + d_2 t^2 + d_3 t^3}, t = \sqrt{\frac{1}{p^2}}$$

$$\begin{aligned} c_0 &= 2.515517 \quad & d_1 &= 1.432788 \\ c_1 &= 0.802853 \quad & d_2 &= 0.189269 \\ c_2 &= 0.010328 \quad & d_3 &= 0.001308 \end{aligned}$$

For the complementary CDF when $0 < p \leq 0.5$. They calculate that the absolute error is below 4.5×10^{-4}. This function approximates Q, such that $Q(x_p) = p$. Since $F(x_p) = 1 - p$, we can swap the values (simply by defining $\tilde{p} = 1 - p$), and obtain the approximation:

$$F^{-1}(p) = G^{-1}(1 - p)$$

This can be used as is for $p > 0.5$, but for values below this threshold, we can take advantage of the symmetry of the Normal distribution, and instead calculate $F^{-1}(p) = -G^{-1}(p)$. Testing confirms this approximation maintains an accuracy greater than 4.5×10^{-10} across $[0, 1]$.

3 Numpy Accelerating the Euler Maruyama Approximation

Implementing this in Python (Listing 1), we can test our standard set of inputs and observe how the results depend on the initial time. In Figure 1, we can see two potential projections. Since the price processes are generated entirely differently

(numpy normal sampling changes based on the number of samples), the processes are independent. We can see that the process when `t0` was 0 had more volatility at the ends and less at the beginning of the period. However, when the process started at 0.5, the process demonstrated more volatility in the centre than the beginning. Both had an upwards trajectory, but neither appears to have finished burning in within 10,000 iterations. Unless we specify that they should follow the same process, they will operate independently.

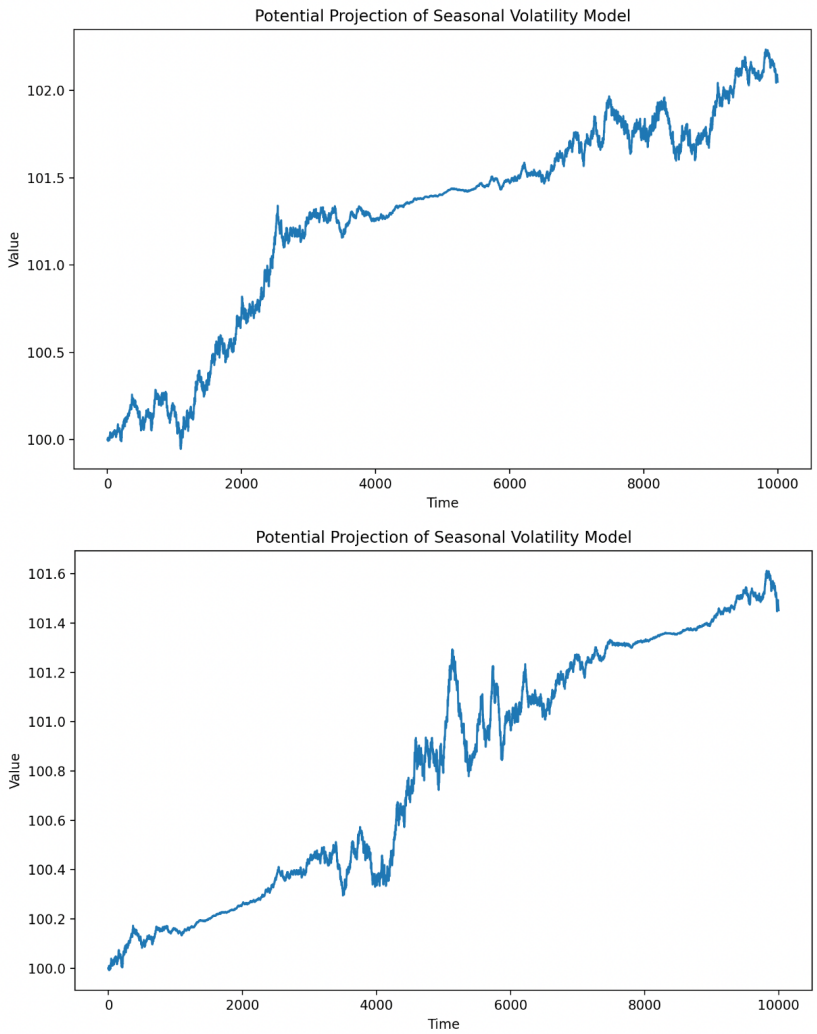

Figure 1: Euler Maruyama price evolution for different initial time values (Top: t0 = 0; Bottom: t0 = 0.5)

3.1 Performance Timing and Accuracy Estimation

We know that $\hat{X} \to X$ as N (or the number of time splits) increases, and $\mu\left[\hat{X}\right] \to \hat{X}$ as we repeat and average the function outputs (increasing M). However, this approach has two Python for loops, which can be slow. This slowness is why we did not achieve convergence within 10,000, requiring a vast amount of samples to get a consistent estimate. The base time complexity of repeating the operation with N timesplits M times is $\mathcal{O}(N \times M)$, which can very quickly become extremely slow if we want to increase M and N for accuracy. We can design a more efficient function for the Euler-Maruyama (EM) model using `numpy`, since `numpy` vectorised mathematics is highly optimised. This has considerable impacts on speed, which we can visualise on a time complexity plot. Observing Figure 3, `numpy` is orders of magnitude faster, completing in seconds what the base prototype does in minutes. Observing Figure 2 we can see that both grow in logged time complexity at linear rates, growing $\mathcal{O}(N \times M)^{\gamma}$. However, we can see the offset and γ are far higher in the base version, indicating a far slower base of calculations and big sample time complexity.

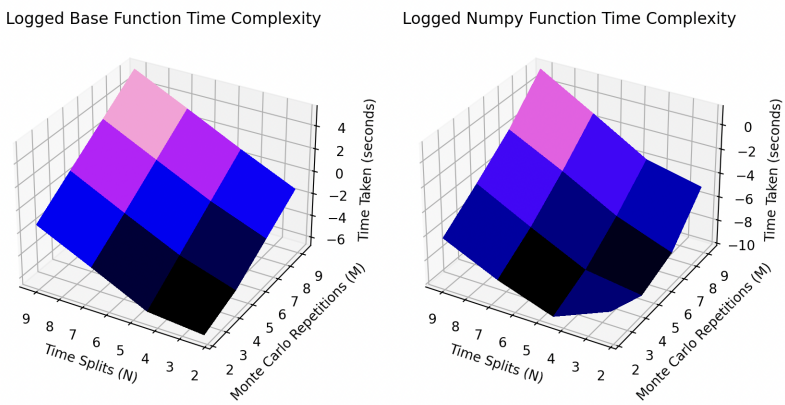

Figure 2: Time Complexity Across `N` and `M` (all values logged)

Of course, high memory and time-efficiency must be measured against accuracy, since, ultimately, the number of iterations required for convergence is key to empirical speed. Observing Figure 4, gains accuracy extremely quickly as time-splits increase. The Euler-Maruyama approximation is a well defined approach, to such an extent that we can identify exactly how many iterations are required to reduce the error. This is because the equation has at least two moments and grows linearly. Sine is a cyclical equation and $0.9\sin(2\pi x)$ can never exceed 1, so the equation grows linearly in t and x. Both are 1-Lipschitz, and consequently continuous on t and x. Thanks to this, we can say that the Euler-Maruyama scheme is strongly convergent in this case, and the accuracy gain from increasing N is non-linear. This makes the efficiency of convergence $\mathcal{O}((\delta t)^{\gamma})$. In order to decrease the error by x times, we can increase our N by $\sqrt{x}$ times. We can observe this empirically in Figure 5, where the absolute error declines exponentially.

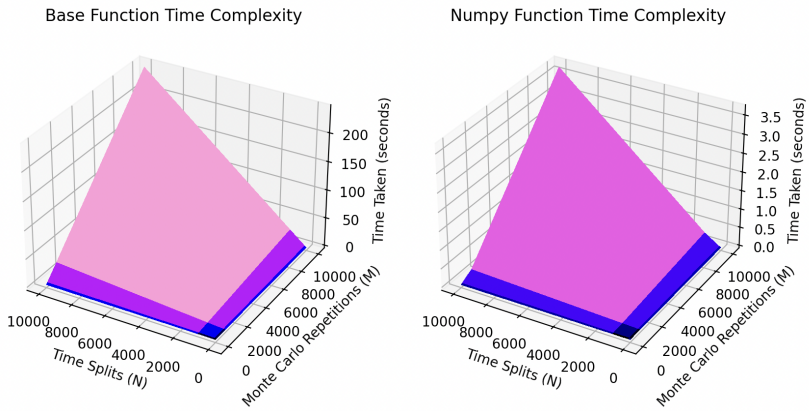

Figure 3: Time Complexity Across `N` and `M` (raw)

4 Defining the Runge-Kutta approximation

The Runge-Kutta (RK) scheme is an alternative to the EM approximation for SDEs that provides strong and weak convergence. This is valuable for path-dependent options which must be evaluated completely. While EM could define an evolutionary process for derivative pricing, we must use RK in order to quickly price options.

For a standard Itô process, RK provides a way of approximating S_{n+1} with:

$$\hat{S} = S_n + a(t_n, S_n)\,\delta t + b(t_n, S_n)\sqrt{\delta t},$$

$$S_{n+1} = S_n + a(t_n, S_n)\,\delta t + b(t_n, S_n)\,\Delta W_t$$
$$+ \frac{1}{2\sqrt{\delta t}}(\Delta W^2 - \delta t)\,[b(t_n, \hat{S}) - b(t_n, S_n)]$$

Unfortunately, in this case, since we have recursive definitions for $\hat{S}$, we are unable to solve this using pure numpy linear algebra. The use of for loops is therefore required to calculate the scheme, increasing time complexity (Listing 2). Due to this increased time complexity, it is important to limit N and M. Running all calls inside functions means that the Python garbage collector deletes all inner function variables when the function goes out of scope, but while commands are running memory needs to be delegated for their calls. In this case, we use $N = 10,000$, and $M = 500$. We can observe from Figure 6 that the burn-in for the sampler finishes around 4,000 steps, and the mean should converge around ≈ 250 chains. Selecting these two options for N and M ensures stable final values through convergence of $\hat{X} \to X$ and $\mu(\hat{X}) \to X$.

```
def price_rk_mc (S_0, mu, sigma, t0, T, N, M):

    dt = T / N
    Declare t as an array of N equal divisions over the period

    Declare S as a vector of M time interval chains
    Set the first value in S to S_0
```

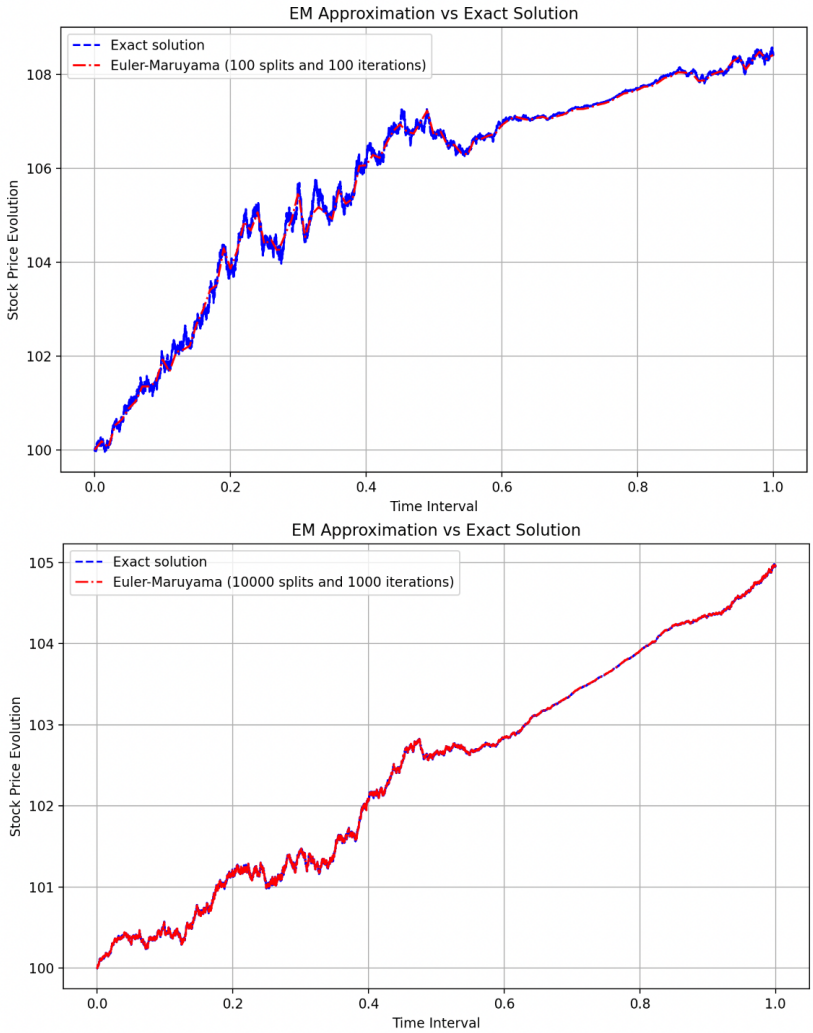

Figure 4: Visualisations of Numpy EM function against exact asset price for the same Brownian Motion process

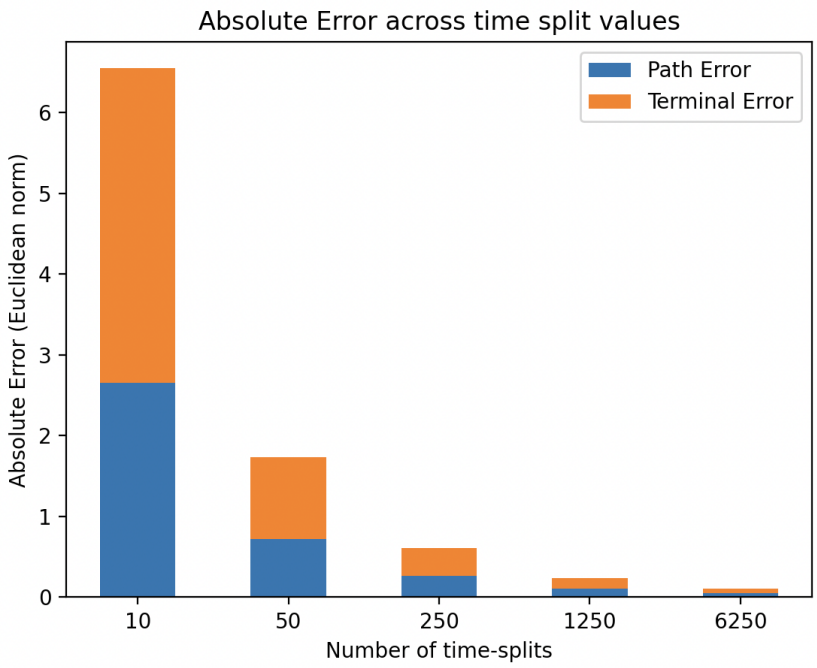

Figure 5: Absolute Error for Euler-Maruyama approximation across time splits

```
for time_steps in t:
    Declare dWt as a scaled vector of standard normal
        samples

    Calculate a, b, S_hat, and b_hat
    Calculate S[n+1]

return S
```

Listing 2: Runge-Kutta option price evolution algorithm

We can calculate the obtained option price across different values of `mu`. Observing Figure 7, we can see that different values of `mu` result in different final prices. The drift factor impacts the convergent value. Since the SDE is a standard Itô process, we know that $\mu(S_T) = S_0 e^{\mu(T-t0)}$. Due to this, if financial institutions were to price options using (estimated) expectations, different prices for the same options could end up being offered. This can create an arbitrage. Consequently, we use the restriction $\mu = r$ to prevent arbitrage.

5 Pricing Exotic Options

Since the RK scheme has weak-convergence, it will be able to efficiently price path-dependent options using Monte Carlo sampling. This is required for efficiently solving options like the Bermudan shout call option. The shout call option mechanically works like a ‘limit‘ order on the regular stock market, where an option is

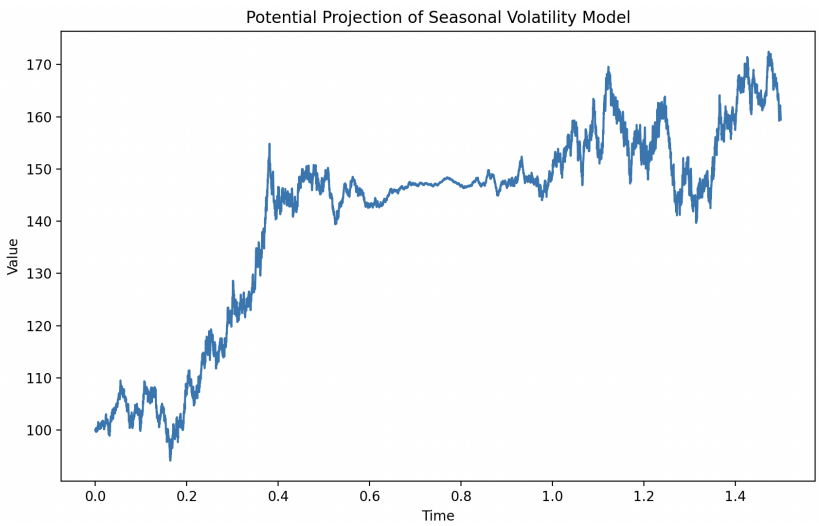

Figure 6: Sample path for Runge-Kutta scheme

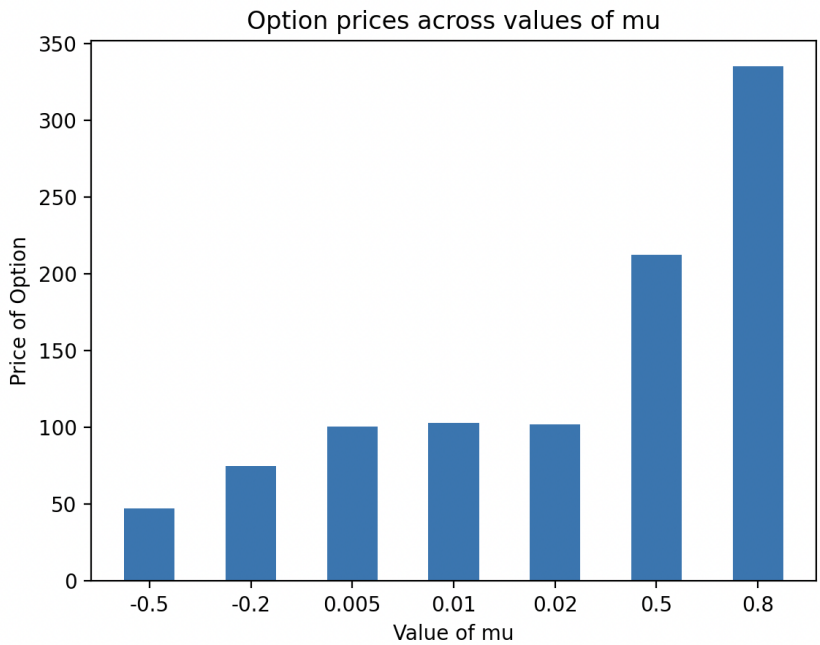

Figure 7: Different prices by `mu`

automatically executed when an offer in the market exceeds the ‘shout price‘. We can price this with the stochastic equation evolutionary process, locking out all paths with values that have exceeded their shout value E, and calculating the terminal payoffs in all paths. The average of these terminal payoffs is a Monte Carlo experimental estimate for the price of the option (Listing 3).

```
def shout_price_rk_mc (S_0, r, mu, sigma, t0, T, K, E, N, M,
    _full_return = False):

    dt = T / N
    Declare t as a time interval over the bounds with N+1 steps

    Declare a vector of current_S values initialised to S_0

    Declare a boolean `shouted` index
    Declare a float `locked` set of values
    Declare an index of shouts for dev returns

    Declare a matrix of paths initialised to S_0

    for time in the time interval:

        take M samples from z
        Calculate the process as before

        identify newly shouting chains

        if there are newly shouting chains:
            calculate your payoff
            index new shouters by time
            index new shouters by chain

        push calculated process onto current_S
        push current_S onto paths

    calculate and return payoffs and Monte Carlo moments
```

Listing 3: Rough pseudo-code for Runge-Kutta option pricing

This can be used to estimate an option price, replete with the Monte Carlo standard error.

```
>> rk_price, std_err = shout_price_rk_mc(S_0 = 100, r = 0.005,
mu = 0.005, sigma = np.sqrt(2*0.005), t0 = 0.0, T = 1.5, K =
    100,
E = 110, N = 50_000, M = 1000)

>> The price of the option is 8.863, and the 95% confidence
intervals are [8.549, 9.177].
```

Listing 4: 95% confidence interval for Bermudan shout option with standard inputs

Listing 4 provides an example output for our default set of inputs when $t_0 = 0$. Since our shout price is not particularly high compared to the initial price, it is not surprising that the price is a number very close to the difference between the initial stock price S_0 and the shout price E. The confidence interval is fairly large, giving us 63 different possible prices (on a two decimal place currency) within the 95% confidence interval.

6 Advanced Monte Carlo to solve path-dependent options

There is a variance reduction technique for Monte Carlo sampling depending on ‘antithetic variates‘. The approach works under the concept that negative covariance reduces standard error. Consequently, if we split our random draws in half, and generate the first half from a parametric distribution like the Gaussian, if we make the second half the opposite value relative to the origin, we will reduce overall variance. This is easily implemented as minor changes in the creation of the `dW` vector and edits to the final calculations, but does come with the stipulation that the number of samples has to be integer divisible by 2. We can compare the outputs of antithetic and standard Monte Carlo sampling on the standard inputs example from Listing 4. Figure 8 makes it clear that the standard error (and confidence intervals) are far lower in antithetic sampling, and in this specific case, the two sampled means are very close together (the base model asserted the price was 9.14 (2 d.p.) and the antithetic function asserted the price was 9.08 (2 d.p.).

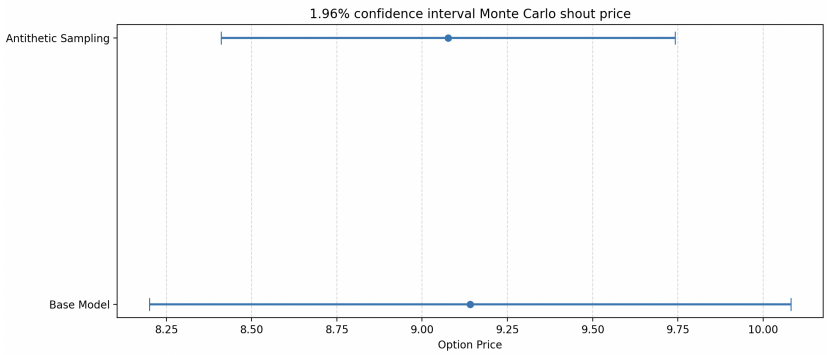

Figure 8: Comparison of estimates between standard Monte Carlo and antithetic variates Monte Carlo